



ПРОГРАММИРОВАНИЕ В ЛИНГВИСТИКЕ

Лекция 8

ОБЪЕКТЫ И КЛАССЫ

Философия ООП. Парадигмы и преимущества

Атрибуты и методы класса. Конструкторы и деструкторы

Наследование. Проверка принадлежности классу

Инкапсуляция и абстракция. Полиморфизм и перегрузка

Философия ООП в Python

- **Объектно-ориентированный ЯП** – все является объектами.
- **Наследование** – каждый объект произведен от класса (типа) и наследует набор его характеристик. Одни классы производны от других.
- **Инкапсуляция** – каждый объект формирует собственное пространство имен. Иерархический доступ к данным объекта.
- **Полиморфизм** – объекты разных классов могут иметь схожие друг с другом интерфейсы. Доступна перегрузка.
- **Абстракция** – при создании класс упрощается до тех характеристик, которые нужны именно в этом коде, а не описывается сразу весь.

Зачем нужно ООП?

СТРУКТУРНЫЙ ПОДХОД

- Разделяет данные и логику
- Иерархически организует код
- Повторное использование кода только через функции и модули
- Лучше подходит для линейных задач – упрощает понимание

ОБЪЕКТНЫЙ ПОДХОД

- Объекты соединяют данные и поведение
- Код организуется вокруг данных
- Повторное использование кода через классы (полиморфизм)
- Подходит для сложных систем

Что такое класс и объект класса?

- **Класс** — это описание свойств и действий для будущих объектов.
- **Объект (экземпляр) класса** хранит данные и свойства, определённые в классе.
- Класс можно представить как **чертёж**, а объект как **дом** по этому чертежу. Один и тот же чертёж позволяет построить **много домов** с разной отделкой и наполнением, но **с общей структурой**.
- Все встроенные **типы данных** являются **классами**!
- При этом все **классы** сами являются экземплярами типа **Object**.

Атрибуты и методы класса

- **Атрибуты** класса – это **свойства**, которыми обладает каждый объект этого класса (**данные**).
- **Методы** класса – это **действия**, которые способен совершать каждый объект этого класса (**операции**).
- И методы, и атрибуты привязаны к **конкретному объекту** и доступны только со ссылкой на него (**self**).
- Эта ссылка нужна, чтобы метод мог обращаться к атрибутам и другим методам **того же объекта** («узнавал» себя).

Определение класса в Python

```
class <ClassName>:  
    <class description>
```

- * Согласно PEP8, имя **класса** записывается в **CamelCase**.
- * Имя **объекта** класса – в **snake_case**, как обычная переменная (**чтобы отличать**).

- Управляющее слово **class** говорит, что перед нами определение класса.
- Одни классы (**дочерние**) можно определять на основе других (**родительских**).
- Тогда в скобках пишем имя родительского класса.

Пример: классы людей и студентов

```
class Human:
    def __init__(self, name):
        self.name = name

class Student (Human):
    def __init__(self, name, group):
        super().__init__(name)
        self.group = group
```

- Дочерний класс Student наследует атрибут name от родительского класса Human.
- При этом Student имеет еще свой атрибут group.
- Метод `__init__` называется конструктором и вызывается при создании объекта класса.

Создание объекта класса

```
vasya = Student('Vasya', 123)
```

- * Создаем объект `vasya` класса `Student`.
- * Значение `'Vasya'` передаем в атрибут `name`, а значение `123` – в атрибут `group`.
- * `type(vasya) == Student`

- При вызове конструктор проверяет, все ли **обязательные поля** заполнены
- Если конструктор задан без параметров, или указано **значение по умолчанию**, тогда при создании объекта можно ничего не передавать.

Удаление объекта класса

```
class Human:
    def __init__(self, name):
        self.name = name
    def __del__(self):
        print(f'{self.name} уничтожен')

vasya = Human('Vasya')
del vasya
```

- Метод `__del__` называется **деструктором** и вызывается автоматически при **удалении** объекта класса из памяти сборщиком мусора.
- Освобождает **ресурсы**, когда счетчик ссылок на объект в коде становится равен 0.
- Можно вызвать командой **del**.

Проверка принадлежности классу

type(obj) → ClassName

- Определяет **точный класс** объекта, **возвращая** его.
- **Не учитывает** иерархию наследования.
- Идеально, когда требуется **строгая типизация**.

isinstance(obj, ClassName) → True/False

- **Проверяет** принадлежность к классу или **любому его подклассу**.
- Поддерживает проверку **нескольких типов** (через кортеж).
- Широко применяется в парадигме **полиморфизма**.

Инкапсуляция атрибутов

- Парадигма **инкапсуляции** – чтобы значения **атрибутов** объекта менялись только через его собственные **методы**.
- Это **защищает** объект от таких изменений, которые не предусмотрены его **логикой**.
- Чтобы инкапсулировать атрибут, **перед его именем** ставим **_**
- Чтобы случайно **не переопределить** имя атрибута в подклассах, можно поставить перед ним **__**, и тогда оно будет автоматически искажаться.

Пример: увеличение пробега машины

```
class Car:
    def __init__(self, brand, mileage, engine_on=False):
        self._brand = brand
        self._mileage = mileage
        self._engine_on = engine_on
    def start_engine(self):
        if not self.engine_on:
            self.engine_on = True
        return "Двигатель запущен."
```

```
def get_mileage(self, distance):
    if self.engine_on:
        self.mileage += distance
        return f"Проехали  
{distance} км, пробег  
{self.mileage} км."
    else:
        return "Двигатель не  
запущен."
```

Абстракция

- Базовые классы создаем с **минимумом атрибутов и методов**.
- Все остальное прописываем непосредственно в наследниках.
- Это нужно, чтобы **не тащить** за собой множество полей для данных, с которыми мы будем работать только в одном из многих случаев.
- При этом **максимально используем наследование**, а не создаем новые базовые классы.
- Логика должна быть **прозрачной**!

Полиморфизм и перегрузка

- Методы базового класса могут быть **переопределены** в классах–наследниках, если требуется **вариативность**.
- Для этого используются **декораторы** – специальные функции, меняющие поведение объектов класса.
- Объявляем декоратор класса перед его определением
- Перед именем декоратора класса ставим **@**
- Декораторы экономят код за счет повторного использования классов.

Пример: простой декоратор класса

```
def simple_decorator(cls): #Модифицируем класс
    cls.new_attribute = "I was added by a decorator"
    return cls
```

```
@simple_decorator
class Example:
    pass
```

```
print(Example.new_attribute)
```

```
# Выведет: "I was added by a decorator"
```

- показывает, что дальше пойдет **описание декорируемого класса**
- К одному и тому же классу можно применять **несколько декораторов**.
- Декорировать можно не только атрибуты, но и **методы класса**.
- Аналогично классам декораторы применяются и **к функциям**.

Главная проблема в ООП

- ООП позволяет оптимизировать дальнейшую **поддержку** разрабатываемого приложения, однако предполагает большую (больше, чем самого кода) роль предварительного **анализа предметной области** и проектирования.
- Одна и та же задача может быть решена **разными объектными моделями**, каждая из которых будет иметь свои плюсы и минусы.
- Только опытный разработчик может сказать, **какую из них** будет проще расширять и обслуживать.

The background is a blue gradient. In the corners, there are white line-art illustrations of circuit boards or neural networks, with lines and small circles representing nodes.

СПАСИБО ЗА ВНИМАНИЕ!